Precision Time Protocol optimization using genetic algorithm

Marta A Plantykow
Intel
Gdańsk, Poland
marta.a.plantykow@intel.com

Milena Olech
Intel
Gdańsk, Poland
milena.olech@intel.com

Maciej Machnikowski
Intel
Gdańsk, Poland
maciej.machnikowski@intel.com

Abstract

This paper represents the work in developing an automated methodology of tuning the Proportional-Integral (PI) controller used by linuxptp applications to even out timing errors in its implementation of the Precision Time Protocol (PTP). In most cases, the hardware-accelerated PTP protocol synchronizes the PTP Hardware Clock (PHC) running on the Network controller. Consequently, system time synchronization requires an additional step that synchronizes the system time to the PHC. PI controller optimization is needed to fulfill the strict phase-synchronization requirement of telecommunication systems introduced by the Long Term Evolution (LTE) E-ULTRA TDD and 5G technologies. This paper explores the use of a Genetic Algorithm (GA), a stochastic algorithm used in Artificial Intelligence (AI), to optimize the parameters of the PI controller. GA mimics the process of natural evolution and has shown to be capable of locating high-performance areas in complex domains. This paper presents different approaches to solve the described problem, various data evaluation methods, and the results achieved in the process.

Introduction

The recent introduction of the Long Term Evolution Time Division Duplexing (LTE-TDD) (3GP) and 5G put rigorous requirements on time synchronization for the telecommunication systems (3GP). This requirement strictly affected Networking hardware implementations. Today, better control of the frequency in a Network Interface Card (NIC) is required to yield much longer holdover times.

Precision Time Protocol (PTP) defines the method of precise synchronization of the time to the grandmaster clock over the Network (Gro20). PTP implementation in linuxptp uses PI controller.

Proportional-Integral-Derivative controllers (PID) were designed for systems that require continuously modulated control. Over 90% of control loops are based on PID controllers (Kno06) (VR12). The quality of a control system based on this technique depends on, among others, PID controller settings.

The problem of choosing correct settings for a controller is continuously analyzed since 1942 (Zie42). Until now, to the best of the authors' knowledge, no universal method of choosing the appropriate settings of the controller was established. Despite having more than 1730 tuning rules defined (VR12), all technicians tune PID controllers using trial and error method (End93). Comprehensive testing of PID implementations indicated that the state-of-the-art control industrial practice report sobering results (End93) - 75% of all PID-based loops are out of tune (UDT98).

Presently Genetic Algorithms (GA), that are stochastic algorithms used in Artificial Intelligence (AI) as search and optimization techniques, have been receiving a lot of attention (MD17) (WST03) (Sko01). GAs proved their effectiveness in the area of Control Engineering, where they were successfully used for PID tuning (MD17) (HBMS02).

GA implementation uses objective functions. The objective function is a mathematical equation that defines the objective of the optimization. Objective functions are used to evaluate solution effectiveness in respect to the optimization goal or, in other words, to award a fitness rating that indicates how successful a particular solution is (MD17) (Obj).

In this work, a framework that is responsible for finding the optimal PID controlling tuning using a Genetic Algorithm is proposed. This paper also covers the analysis of possible data evaluation methods for the optimization goal and goes over the results of the proof of concept work.

The paper is divided into four main subsections.

The Related work section presents the analysis of Proportional-Integral-Derivative controller design, introduces PID Tuning process, basic PID Tuning methods and PID Optimisation using Genetic algorithms description. It describes the basics of Precision Time Protocol and PTP on Linux implementations. In the end of this section possible Data evaluation methods are described.

The second section describes Our approach. This section is divided into subsections that describe the Developed framework details, chosen Data evaluation methods and research Results.

The third section describes Conclusions.

Finally, the last section describes the Future Work.

Related work

Precision Time Protocol (PTP) is an industry standard protocol used for network clocks synchronization (Gro20). The linuxptp implementation of the PTP protocol uses Proportional-Integral-Derivative (PID) controllers. Quality of the systems based on PID controllers depends, among others, on PID controllers settings. A lot of research tackled the problem of finding the proper values for PID settings (VR12). In 2009 Aidan O'Dwyer presented the results of his work on summarising available controllers tuning methods (over 1730) (VR12). Brain Wave found that 75% of all PID controllers used in the industry are out of tune (UDT98). Genetic Algorithms (GA) have proven to be capable of locating high performance areas in complex domains, including finding optimal settings for PID controllers. (MD17) (HBMS02) (WST03) The presence of many data evaluation methods allows for a direct comparison.

Precision Time Protocol

PTP is the IEEE 1588 standard. The standard defines the method of precise synchronization of the time to the grandmaster clock over the Network. The protocol supports system-wide synchronization in the submicrosecond range with minimal requirements on network and local computing resources. The clocks within a system are organized into a leader-follower hierarchy. The clock at the top of the hierarchy determines the reference time for the entire system. The standard provides administration-free operation and allows simple systems to be installed and operated without administrative attention. The protocol applies to both high-end and low-end devices. The protocol operates on the principle of exchanging messages between the Leader and the follower. (Gro20)

The main messages are Sync, FollowUp, DelayReq, and DelayResp. The Leader broadcasts Sync message. Its arrival is timestamped at the follower side. The next message is the FollowUp message containing the timestamp of when the packet left the Leader. Based on those two messages, the follower can calculate the offset between its timer and the Leader's one. The subsequent two messages are used for measuring the path delay. (Gro20)

The follower sends the DelayReq message and saves the time when it left the follower. The leader timestamps received DelayReq messages and sends that timestamp back to the follower using the DelayResp message. The follower can calculate the offset between two clocks and the path delay compensation with all four timestamps. (Gro20)

In terms of accuracy it's better than the Network Time Protocol (NTP) ($\sim$ 100ms) and worse than the Global Navigation Satellite System (GNSS) $\sim$30-40 ns (Gal2C), (GPS2C).

PTP on Linux

The most commonly used PTP implementation on Linux is the linuxptp toolset. It consists of three main tools:

- ptp4l is the tool that implements the IEEE 1588 protocol. This tool will synchronize the PTP Hardware Clock (PHC) inside the Network Interface Card (NIC) to the Grandmaster time in a hardware-accelerated case. In a software implementation it synchronizes the system clock to the time obtained from the Grandmaster.
- ts2phc can be used to synchronize the PHC clocks to the source of the external events, like the 1 Pulse per Second (PPS) signal coming from the Global Navigation Satellite System receiver
- phc2sys synchronizes two clocks - for example, it can synchronize the system clock to the PHC inside the NIC, and the other way around (phc).

Proportional-Integral-Derivative

PID controllers were designed for systems that require continuously modulated control. They are the most widely used control mechanisms that are based on the idea of a feedback loop. It is estimated that over 90% of control loops employ such solution (Kno06) (VR12). The application of PID controllers ranges from small industry to high technology industry. PID controllers can be used in many forms, such as a stand-alone controller, part of Direct Digital Control package, or even Distributed Control System (MD17).

The quality of a control system based on this technique depends, among others, on PID controllers settings.

Figure 1 presents a simplified block diagram of an ideal PID controller. The controller is a combination of three basic building blocks: proportional, integral, and differential.

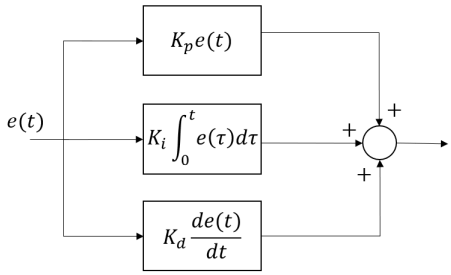


Figure 1: Simplified block diagram of an ideal PID controller

Input signal marked on a Figure 1 as $e(t)$ represents the error, that will be described in following section.

The proportional term, that multiplies error signal by proportional gain (K_p) responses immediately to the current error value (Kno06). Proportional term transfer function is described in equation 1.

$$p(t) = K_p * e(t) \tag{1}$$

The integral term, that multiplies the integral of an error by integral gain (k_i) sums the error over a time. Integral

term transfer function is described in equation 2.

$$i(t) = K_i * \int_0^\tau e(\tau)\,\mathrm{d}\tau \tag{2}$$

The derivative response multiplies the rate of change of the process variable by derivative gain (k_d) (PID). Proportional term transfer function is described in equation 3.

$$d(t) = K_d * \frac{de(t)}{dt} \tag{3}$$

To properly control the system using a PID controller previously mentioned settings: K_p, K_i and K_d must be found. Possible methods to adjust controller parameters are analyzed in subsection PID Tuning methods.

Every PID controller operates in a system. Figure 2 presents a simplified block diagram of an ideal PID controller when applied in a system.

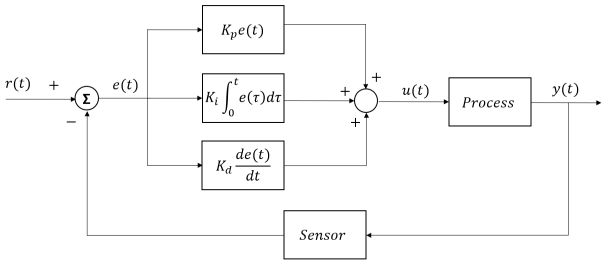


Figure 2: Simplified block diagram of an ideal PID controller in a system

Input signal, marked on a Figure 2 as $r(t)$ represents the value that the control system is aiming to reach. This value is referred to as a setpoint in some sources. The response of a system is described as $y(t)$ on the Figure 2 is measured by the sensor and (assuming having an ideal sensor) is equal to the value the control system reached at a given point of a time. Error level is calculated using the feedback loop and is described as $e(t)$ represents the difference between the value that the system is trying to reach and the value reached by the system. It is described in equation 4.

$$e(t) = r(t) - y(t) \tag{4}$$

Signal coming out of the PID controller $u(t)$ is a steering signal (the one that interfere directly with the process). It is described in equation 5.

$$u(t) = p(t) + i(t) + d(t) \tag{5}$$

Due to the lack of industry standards, nine different PI/PID controllers structures have been defined in literature (VR12). The example presented in this section describes only the ideal PID controller.

A bathtub can be used as an example of a system that uses a PID controller. The goal of a given control system is to keep the level of the water in the tub at a fixed level $r(t)$. The level of the water in the tub at a given point of time is described as $y(t)$. The Error level (described by the equation 4) represents the missing amount of water or its overflow. In this example, a PID controller can control the water flow from the tap.

PID Tuning

Historically, the first tuning rule for setting up controller parameters was defined and patented by Albert Callender, and Allan Brown Stevenson in 1934 (CA34). In this work, the Proportional-Derivative (PD) controller for a process was modeled using an integrator plus delay (IPD) model. A few years later, in 1937, PI and PID controllers tuning rules were defined. In this case, there was an assumption that the process was precisely modeled by a First-Order Lag Plus Delay model or a pure delay model (VR12).

David Ender conducted control loop testing in hundreds of operating plants. The reported results indicated that 75% of all PID-based loops are out of tune (UDT98).

> In practice, all technicians teach themselves to tune by the seat of the pants, sometimes referred to as trial and error or SWAG tuning. It typically involves "tweaking" the settings until the loop "looks" right. (End93).

Every auto-tuning procedure starts by taking input/output measurements (in open or closed loops). There are two types of paths that can be obtained when tuning the PI/PID controller. The first uses measurement results to determine the regulator parameters, while the second uses the same measurements to obtain a process model. Process model permits simulation of the closed-loop system virtually, and always takes the form of a transfer function which can later be used to tune the controller (VR12).

Both approaches have their advantages and disadvantages. Having a model of the system makes it possible to simulate its answer before applying the tuned regulator to an actual process (VR12). On the other hand, the process of model creation may be expensive. Also, the process of model creation may have to be repeated frequently, as the transfer function representing the system changes in its life cycle.

PID Tuning methods

The main problem with PID controllers usage arises when determining the value of K_p, K_i and K_d (MD17). In the last century, a large number of researchers considered PID controllers and their settings, improving their functionality (VR12). The handbook of PI and PID controller Tuning rules (3rd edition) written by Aidan O'Dwyer (2009) presents over 1,730 controller tuning methods. 563 of them were defined for PI controllers, and 1,168 rules for PID controllers (VR12). Unfortunately, most classical methods require the explicit mathematical modeling of the whole processing plant (MD17).

Possible PID tuning techniques are based on one of the three groups of methods (HBMS02):

- Empirical methods such as Ziegler and Nichols 1942 (Zie42)
- Analytical methods such as Asrom and Hagglung 1995 (KJA95)
- Methods based on optimization such as Cian-cone or Lopez methods (HBMS02)

All described tuning techniques types require the model of an object to be considered linear. This means that the tuning is not optimal when a process does not operate in its validity zone. This usually happens when the goal of the system is not to reach a particular set point but to follow a trajectory (for example, heating systems) (HBMS02).

As a solution to this problem, a separate model for each operational zone can be used. In this case, the PID controller must be tuned for each model, and a mechanism for changing one controller to another depending on the operation zone must be implemented.

Another alternative is to tune a PID controller by using Genetic Algorithms. This method takes into account all non-linearities and additional process characteristics.

PID Optimisation using Genetic algorithms

GAs are considered random search methods that mimic natural evolution and have been shown to locate high-performance areas in complex domains.

GAs depends entirely on responses from the environment, and to reach the best solution, they use evolution operators such as reproduction, crossover, and mutation, mimicking the process of natural evolution (MD17).

The procedure starts with a randomly generated group of possible solutions called population, in which each object in the population is called a chromosome. Every created chromosome is evaluated using the objective function (more details in section Data evaluation methods), which assigns each individual a corresponding number called its fitness. The best chromosomes are selected from the population to create a new generation using crossover and mutation operations. In the last step, a new generation replaces the initial one. The procedure repeats until an optimal population is found or a specified time passed. (MD17)

Figure 3 presents a simplified block diagram of the GA.

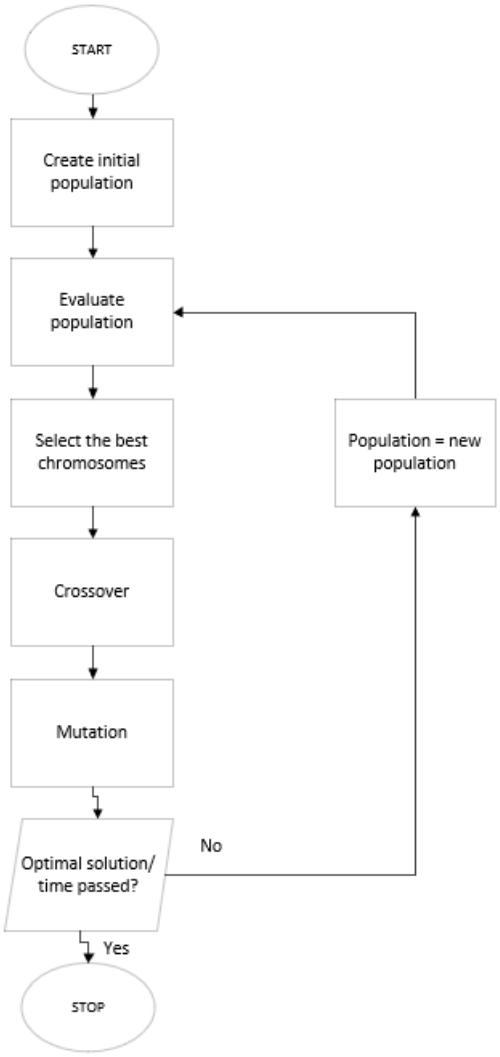


Figure 3: Simplified block diagram of the Genetic Algorithm

Data evaluation methods

To compare expected values $(P_i, i = 1, 2, ..., n)$ and observations $(O_i, i = 1, 2, ..., n)$ dimensions measures of average error was used. In this approach model errors are defined as $e_i = P_i - Oi$. Measures of the average error that depicts model performance are based on summaries of e_i $(i = 1, 2, ..., n)$. In general average model error can be represented by formula 6.

$$e_\gamma^- = [\sum_{i=1}^{n} w_i |e_i|^\gamma / \sum_{i=1}^{n} w_i]^{\frac{1}{\gamma}} \quad (6)$$

where $\gamma >= 1$ and w_i is a scalar weight assigned to e_i according to its impact on the entire error. In this paper w_i is assumed to be equal 1. Average error is most commonly used with $\gamma = 2$. This represents Root Mean Squared Error - $RMSE$ that described by the formula 7.

$$RMSE = [n^{-1} \sum_{i=1}^{n} |e_i|^2]^{\frac{1}{2}} \quad (7)$$

The sensitivity of $RMSE$ is caused by the fact that each error e_i is squared, what strives for greater influence on the final result. When γ is altered to 1, average error becomes Mean Absolute Error - MAE, that is described in formula 8.

$$MAE = n^{-1} \sum_{i=1}^{n} |e_i| \quad (8)$$

This measure sign the magnitude of the errors by applying absolute value. Provided that $\gamma = 1$ and the signs of the errors are taken into account, the average error has been defined as Mean Bias Error - MBE. It is described in formula 9.

$$MBE = n^{-1} \sum_{i=1}^{n} e_i \quad (9)$$

Nonetheless the MBE value is mostly considered to indicate average model bias rather than provide error magnitude.

Values of γ other than 1 or 2 may be opposite in particular situations, but $\gamma = 1$ and $\gamma = 2$ have essential advantages. Values of γ different than 1 represent weighting of each e_i. In the case of $\gamma = 2$, e_γ^- and e_i are presented in the format similar to variance, which enhances further mathematical and statistical analysis.

Calculation of MAE consists of two parts. The first part is summing absolute values of the errors, then dividing the total error by n. Calculation of RMSE involves three steps. First, total square error is obtained as a sum of individual squared errors. This means that large errors have relatively greater influence on the total than smaller errors. Subsequently, total square error is divided by n, which is equal to Mean Squared Error - MSE. The last step is to take RMSE as the square root of the MSE.

Our approach

This work aims to optimize the time synchronization using the linuxptp suite by optimizing the Proportional-Integral (PI) servo's parameters. The proposed framework automatically searches for the best variables using a Genetic Algorithm (GA). There is a wide range of possible optimization goals, and different evaluation functions can be used to fulfill each goal.

Developed framework details

The framework we developed is highly flexible and accepts many parameters. Parameters which determine the interface to work on *--i* must be determined when running the script. Other parameters will be set to the default values if not used. Table 1 contains the full list of available parameters and it's default values.

The procedure starts with a randomly generated group of chromosomes called population. Initial group size is determined by the *--pop_ size* argument. Each chromosome is described by two values: K_p and K_i. The maximal value of randomly generated K_p and K_i for each chromosome is controlled by *--max_ kp* and *--max_ ki* arguments. Minimal value equals 0.

Every chromosome in the population has to be evaluated. Chromosome evaluation process starts with the PI controller, where parameters change to values that the chromosome is described by. In the next step a phc2sys test is executed on the interface described by the *--i* parameter and it lasts for as many seconds as determined using *--t* parameter. As a result of this operation an output file that consists of following information is created:

- Controlled object name
- Source clock
- Offset that represents the difference between source and target clock
- Controller state (s0 - clocks not synchronised, s1 - there is a big difference between source and target clock, s2 - only frequency of the controlled object is changed)
- Frequency difference to the master clock (in Parts Per Billion (PPB))
- Delay of the path

The value of the offset returned from the test is used to calculate and assign the fitness value by using one of the evaluation metrics that is determined by the *--metric* parameter. Fitness value is a number that determines how successful a given chromosome was. As a default the MSE metric will be used. If determined, RMSE and MAE metrics can be also used.

In the next step the best chromosomes are selected from the population to create the new generation. Number of chromosomes that are going to be crossed with each other (combination) is determined by the *--num_ inherited* parameter. Additionally, the number of the best chromosomes determined by the *--num_ replicated* parameter is directly copied to the new generation. To increase the exploratory nature of an algorithm another parameter

Table 1: Framework parameters

Parameter	Description	Def
--pop_ size	Initial population size	8
--epochs	Number of epochs	10
--max_ kp	Max value of K_p	15
--max_ ki	Max value of K_i	15
--num_ random	Number of random parents added to each epoch	2
--num_ inherited	Number of the best parents that are crossed to create a new generation	5
--num_ replicated	Number of the best parents that are replicated to create a new generation	4
--mutation_ coef	Mutation coefficient	1
--debug_ level	Determines level of debug prints	1
--i	Interface name	-
--t	Time of a single test	120
--metric	Evaluation metric	1
--elite_ size	Number of elite chromosomes	1

called --*num_ random* may be used to add into consideration an additional number of chromosomes that are generated randomly.

When the new population is created the mutation operator is introduced. The main goal of its existence in the algorithm is to increase the population diversity, ensure that the GA searches the entire problem space and that it didn't find the sub-optimum due to a bad choice of initial population (MD17). Mutation is defined as a occasional random alteration of a chromosome. The influence of it is determined by the --*mutation_ coef* parameter.

Crossover and mutation introduces a high probability of losing the optimum solution. There is no guarantee that the fittest chromosome will be saved for the next generation. To avoid this, the elitist is used. A number of the best individuals from a population regulated by the --*elite_ size* parameter is saved before any of these operations take place. When a new population is formed and evaluated, this model will examine to see if there are any better results and update its content accordingly.

When the mutation process finished successfully whole process repeats and every chromosome in the population is evaluated again. The number determined by the parameter --*epochs* determines how many times the whole procedure is executed.

A simplified block diagram that shows the framework workflow is presented in the figure 4.

During execution the proposed framework creates a log file. It contains K_p, K_i and the fitness function value of:

- The best result reached in every epoch,
- Elite
- Default phc2sys PI controller settings

Data evaluation

The primary goal of this paper is to make our follower system's time matched the leader time as closely as possible. That is why we tune K_p, and K_i - to minimize the leader offset.

In an ideal scenario, the offset between the leader and our system tune is equal to 0. Evaluation methods are needed to calculate the difference between expected and observed values. The developed framework gives a possibility to choose evaluation metrics. Three options are the most accurate from the statistical point of view: MSE, RMSE, and MAE.

Moreover, evaluation methods let us compare leader offset error generated by default K_p and K_i and leader offset error created by values of K_p, and K_i returned from GA.

Results

A large set of experiments to optimize the PI controller parameters implemented in Linux PTP was conducted by the team. Presented results describe three experiments in detail. In each experiment, phc2sys test response with PI controller tuned with values returned by GA was

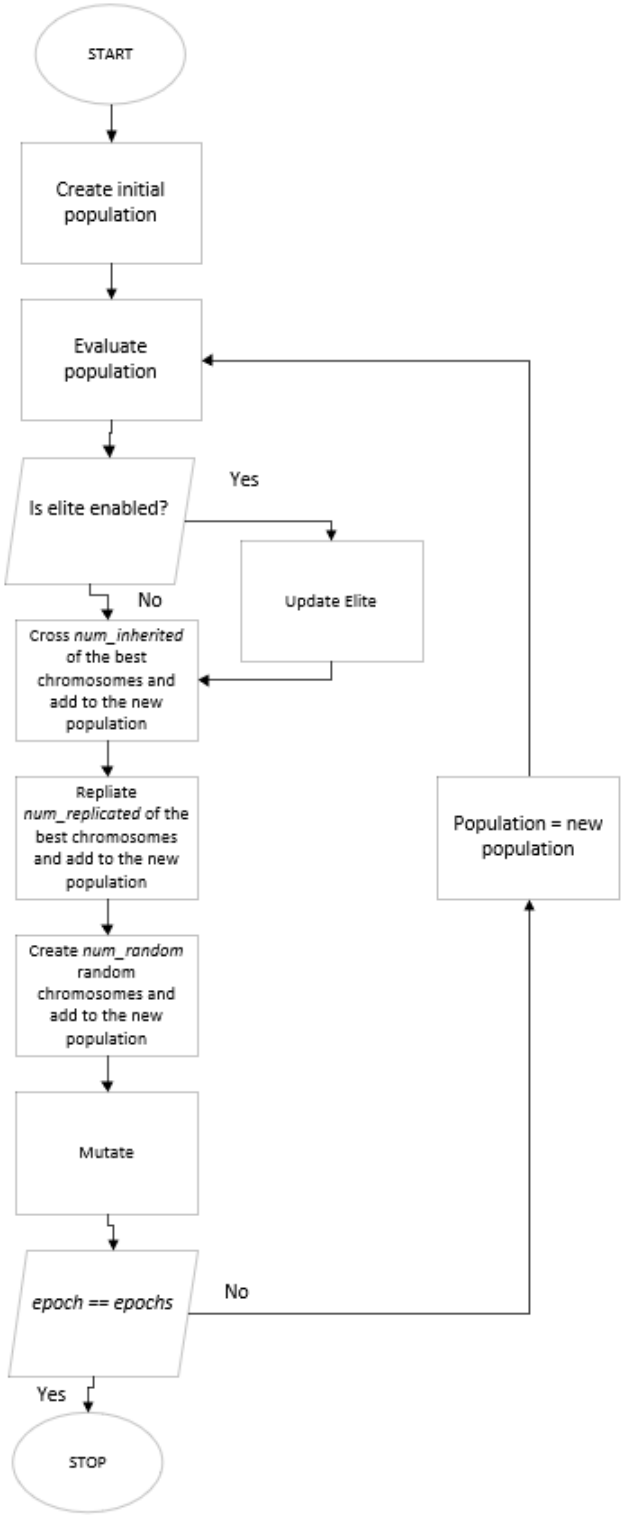


Figure 4: Simplified block diagram of the Genetic Algorithm implementation

compared with phc2sys test response with PI controller tuned with default settings ($K_p = 0.7$ and $K_i = 0.3$).

Each experiment result is presented as a table and two sets of figures. Each table contains PI controller settings (elite and default), the value returned by the metric, and calculated improvement. The first set of figures for each experiment shows phc2sys test results ran with PI controller settings indicated by the GA and its histogram. The second set of figures presents the phc2sys test result ran with default PI controller settings and histogram. The histogram is used to present values distribution to analyze how often each different value occurs in a data set.

The framework on the device under test was run with *--max_kp* = 5, *--max_ki* = 5, *--t* = 80, other parameters were set to default values listed in Table 1. The experiment was repeated for each metric - MSE, $RMSE$, MAE.

GA results using Mean Squared Error (MSE) as a metric are presented in Table 2.

Table 2: GA results using MSE as a metric

	K_p	K_i	MSE
Default	0.7	0.3	110.26
Elite	0.59	0.93	75.08
		Improvement	**32%**

GA proved to be capable of finding PI controller settings that return 32% smaller MSE error when running the phc2sys test.

Figures 5 and 6 presents the results of phc2sys tests ran with PI controller settings indicated by the GA using MSE as a metric and with default PID settings.

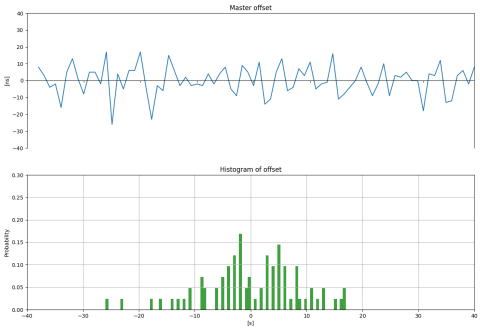


Figure 5: Results of phc2sys tests ran with PI controller settings indicated by the GA ($K_p = 0.09$, $K_i = 0.59$) using MSE as a metric

The histogram visibly shows that values returned by the phc2sys test with PI controller tuned with values returned from GA (Figure 5) are more centered around 0 than in the case of the other test (Figure 6).

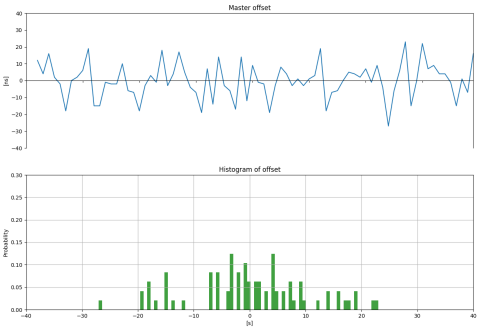


Figure 6: Results of phc2sys tests ran with default PID settings ($K_p = 0.7$, $K_i = 0.93$) using MSE as a metric

GA results using Root Mean Squared Error (RMSE) as a metric are presented in Table 3.

Table 3: GA results using RMSE as a metric

	K_p	K_i	RMSE
Default	0.7	0.3	11.68
Elite	0.09	0	8.98
		Improvement	**23%**

GA proved to be capable of finding PI controller settings that return 23% smaller RMSE error when running the phc2sys test.

Figures 7 and 8 presents the results of phc2sys tests ran with PI controller settings indicated by the GA using RMSE as a metric and with default PID settings.

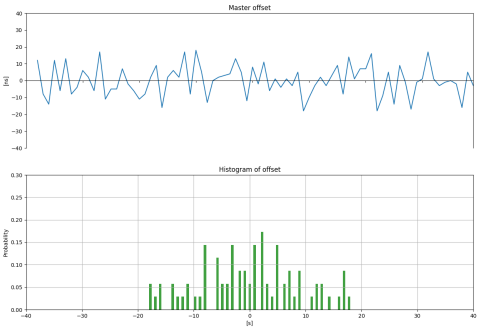


Figure 7: Results of phc2sys tests ran with PI controller settings indicated by the GA ($K_p = 0.09$, $K_i = 0$) using $RMSE$ as a metric

The histogram visibly shows that values returned by the phc2sys test with PI controlled tuned with values returned from GA (Figure 7) are more centered around 0 than in case of the other test (Figure 8).

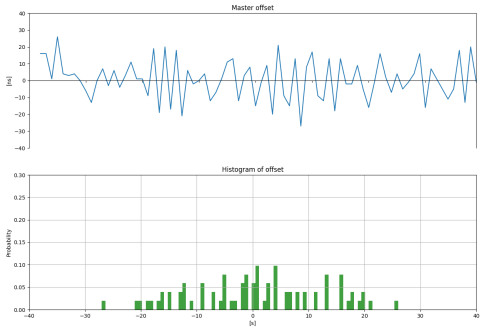


Figure 8: Results of phc2sys tests ran with default PID settings ($K_p = 0.7$, $K_i = 0.93$) using $RMSE$ as a metric

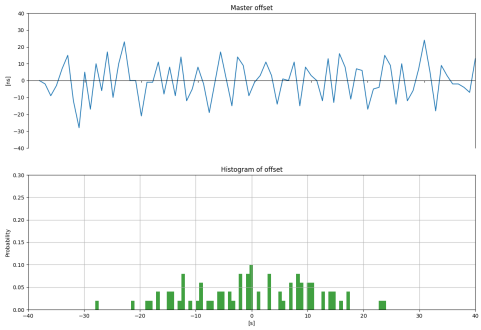


Figure 10: Results of phc2sys tests ran with default PID settings ($K_p = 0.7$, $K_i = 0.93$) using MAE as a metric

GA results using Mean Absolute Error (MAE) as a metric are presented in Table 4.

Table 4: GA results using MAE as a metric

	K_p	K_i	MAE
Default	0.7	0.3	8.99
Elite	0	2.51	7.32
		Improvement	**19%**

GA proved to be capable of finding PI controller settings that return 19% smaller MAE error when running phc2sys test.

Figures 9 and 10 presents the results of phc2sys tests ran with PI controller settings indicated by the GA using MAE as a metric and with default PID settings.

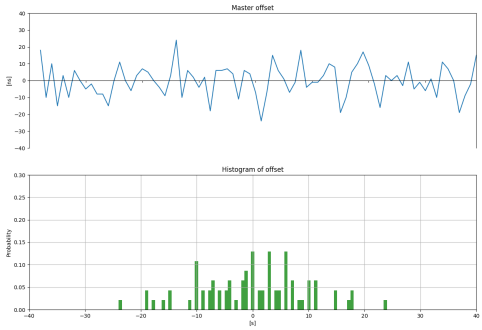


Figure 9: Results of phc2sys tests ran with PI controller settings indicated by the GA ($K_p = 0$, $K_i = 2.51$)using MAE as a metric

The histogram visibly shows that values returned by the phc2sys test with PI controlled tuned with values returned from GA (Figure 9) are more centered around 0 than in the case of the other test (Figure 10).

In our experiments, we used different general average model error metrics to optimize various aspects. MSE, RMSE, and MAE metrics show average model error in units of the variable of interest, and expected results range from 0 to ∞. All of them are negatively oriented, which means that the lower the value obtained, the better model performance.

RMSE and MSE share most properties because RMSE is the square root of MSE. Nonetheless, MSE is more biased for higher values, while RMSE is more efficient in reflecting model performance when large error values are considered. Considering that in the event of RMSE, errors are squared before they are averaged, the RMSE gives a relatively high weight to significant errors. In other words, **RMSE should be more useful when significant errors are unwelcome**.

When the overall impact is proportionate to the actual increase in error, then it is better to calculate MAE. From an interpretation point of view, if the error value increases to 12 from 6, the actual influence on the result is twice (in the case of RMSE and MSE is more than twice).

In our research, GA has achieved a more significant improvement using RMSE and MSE than MAE. This observation can be explained by the fact that our purpose was to account for outliers in general average model errors. Hence, a crucial part of the data evaluation process was penalization for significant errors.

Conclusions

The developed framework provides an easy-to-use automated methodology of tuning the Proportional-Integral controller embedded in the linuxptp project. Default parameters used by phc2sys usually yield "good enough" results. However, in all tested cases, the developed framework was able to deliver better results. All metrics used throughout this paper proved that the synchronization quality depends on linuxptp's PI controller parameters.

Most runs required only a couple of generations to find parameters that were significantly better than the defaults.

By replacing the data evaluation method, the same framework can optimize different aspects of time synchronization.

Future Work

Future research on Precision Time Protocol optimization using a Genetic Algorithm could fruitfully explore a coarse validation of the stability of input parameters. According to John C. Edison (Eid06) the stability properties of the controller can be described by Equation 10, Equation 11 and Equation 12. Considering this, all configurations that are unstable by definition could be removed from consideration to optimize the framework performance and eventually speed it up.

$$(K_p + K_i)^2 = 4K_i \tag{10}$$

$$0 \leq K_i \leq 4 \tag{11}$$

$$0 \leq K_p \leq 1 \tag{12}$$

Furthermore, we believe that apart from looking for Phc2sys related solutions, future research should look for PI controller settings for ptp4l as defined in the IEEE 1588 standard.

In future work, investigating longer test runs might deliver better results.

In addition, finding an optimal number of epochs and optimal elite size might prove an important area for future research.

Acknowledgments

We'd like to acknowledge the assistance of Jesse Brandeburg for his invaluable support and engagement throughout creating this paper.

References

[3GP] 3GPP. Evolved universal terrestrial radio access (e-utra); requirements for support of radio resource management. `https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=2420`. Accessed: 2021-6-9.

[CA34] Brown A. Callender A. Automatic control of variable physical characteristics, U.S. Patent 2 175 985, 1934.

[Eid06] J. Eidson. Measurement, control, and communication using ieee 1588. 2006.

[End93] D. Ender. Process control performance : Not as good as you think. *Control Engineering*, 24, 1993.

[Gal2C] Communication from the commission to the european parliament and the council - state of progress of the galileo programme. *Official Journal of the European Communities*, 242(2), 2002/C.

[GPS2C] Global positioning system standard positioning service performance standard. `https://www.gps.gov/technical/ps/2020-SPS-performance-standard.pdf`, 2002/C. Accessed: 2021-6-9.

[Gro20] Working Group PNCS Precise Networked Clock Synchronization Working Group. Ieee standard for a precision clock synchronization protocol for networked measurement and control systems. *IEEE Std 1588-2019 (Revision of IEEE Std 1588-2008)*, pages 1–499, 2020.

[HBMS02] H. M. Herrero, X. Blasco, M. Martinex, and J.V. Salcedo. Optimal pid tuning with genetic algorithms for non linear process models. *15th Triennial World Congress, Barcelona, Spain*, 2002.

[KJA95] Tore Hagglund Karl J. Astrom. *PID Contollers, 2nd Edition*. 1995.

[Kno06] C Knospe. Pid control. *IEEE Control Systems Magazine*, 2006.

[MD17] D. C. Meena and Ambrish Devanshu. Genetic algorithm tuned pid controller for process control. In *2017 International Conference on Inventive Systems and Control (ICISC)*, pages 1–6, 2017.

[Obj] Objectives and constraints. `https://abaqus-docs.mit.edu/2017/English/SIMACAEANLRefMap/simaanl-c-optobjectives.htm`. Accessed: 2021-7-1.

[phc] phc2sys. Accessed: 2021-7-1.

[PID] Pid theory explained. `https://www.ni.com/en-ie/innovations/white-papers/06/pid-theory-explained.html`. Accessed: 2021-6-8.

[Sko01] Sigurd Skogestad. Probably the best simple pid tuning rules in the world. 01 2001.

[UDT98] Universal dynamic technologies: Brainwave: The new concept in process control, 1998.

[VR12] A. Vilanova R., Visioli. *PID Control in the Third Millennium*. 2012.

[WST03] Qing Wang, P. Spronck, and R. Tracht. An overview of genetic algorithms applied to control engineering problems. In *Proceedings of the 2003 International Conference on Machine Learning and Cybernetics (IEEE Cat. No.03EX693)*, volume 3, pages 1651–1656 Vol.3, 2003.

[Zie42] Nichols N.B. Ziegler, J.G. Optimum settings for automatic controllers. *IEEE Control Systems Magazine*, 64:759–768, 1942.